

Introdução a Grafos — Parte II

Universidade Estadual do Sudoeste da Bahia - UESB

2026.1

Árvores — Definições Básicas

Um grafo é uma **árvore** se e somente se satisfaz qualquer uma das condições equivalentes abaixo:

- ▶ For **conexo** e possuir N vértices e $N - 1$ arestas.
- ▶ For **conexo** e não contiver ciclos.
- ▶ Cada par de nós possuir exatamente **1 caminho** entre si.
 - For conexo e, ao retirar qualquer aresta, tornar-se desconexo.
 - Não contiver ciclo e, ao adicionar qualquer aresta, ganhar um novo ciclo.

Árvores — Terminologia

- ▶ **Folha:** vértice com grau 1 (apenas um vizinho).
- ▶ **Raiz:** vértice de uma árvore *enraizada*, ou seja, aquela cujos vértices possuem níveis de profundidade determinados pela distância até a raiz.
- ▶ **Pai:** o pai de x é o vértice imediatamente mais próximo da raiz que x . A raiz não possui vértice pai.
- ▶ **Filho:** o filho de x é o vértice imediatamente mais distante da raiz que x .
- ▶ **Ancestrais:** o conjunto formado pelo pai de x e pelos ancestrais desse pai.
- ▶ **Subárvore:** o conjunto de todos os nós que possuem x como ancestral.

Obs.: o **grau** de um vértice é o número de seus vizinhos.

Problema 1 — Subordinates

Enunciado: Dada a hierarquia de uma empresa com n funcionários, calcule para cada funcionário a quantidade de seus **subordinados** (diretos e indiretos). O funcionário 1 é o diretor geral.

Entrada:

```
5
1 1 2 3
```

(os chefes diretos dos funcionários 2, 3, 4 e 5)

Restrições: $1 \leq n \leq 2 \cdot 10^5$

Saída:

```
4 1 1 0 0
```

(quantidade de subordinados de cada funcionário)

Percorrendo Árvores — Tamanho de Subárvore

Com a DFS, é possível calcular **recursivamente** o tamanho de cada subárvore. O tamanho da subárvore de x é 1 (o próprio x) mais a soma dos tamanhos das subárvores de seus filhos.

```
vector<int> subtreeSize(N, -1);

void dfs(int x) {
    subtreeSize[x] = 1;
    for (int u : adj[x]) {
        if (visited[u]) continue;
        visited[u] = true;
        dfs(u);
        subtreeSize[x] += subtreeSize[u];
    }
}
```

Hora de Implementar

Subordinates

Implemente a solução utilizando DFS e tamanho de subárvore.

Problema 2 — Tree Diameter

Enunciado: Dada uma árvore com n nós, determine o seu **diâmetro**, ou seja, a maior distância (em número de arestas) entre dois nós quaisquer.

Entrada:

```
5
1 2
1 3
3 4
3 5
```

Saída:

```
3
```

(caminho: $2 \rightarrow 1 \rightarrow 3 \rightarrow 5$)

Restrições: $1 \leq n \leq 2 \cdot 10^5$

Percorrendo Árvores — Profundidade

A **profundidade** do vértice x é a sua distância até a raiz.

Percorrendo Árvores — Profundidade

A **profundidade** do vértice x é a sua distância até a raiz.

```
vector<int> depth(N, -1);

void dfs(int x) {
    for (int u : adj[x]) {
        if (visited[u]) continue;
        visited[u] = true;
        depth[u] = depth[x] + 1;
        dfs(u);
    }
}
```

Problema do Diâmetro — Solução Gulosa

O diâmetro pode ser encontrado com uma estratégia gulosa simples em **2 DFS**:

- 1º Escolhe-se um vértice qualquer da árvore (s).
- 2º Encontra-se o vértice mais distante de s — chamado x .
- 3º Encontra-se o vértice mais distante de x — chamado z .
- 4º O diâmetro é a distância de x até z .

Prova intuitiva: pode-se demonstrar que x é sempre uma extremidade de algum caminho de comprimento máximo na árvore. Logo, a maior distância a partir de x é necessariamente o diâmetro.

Hora de Implementar

Tree Diameter

Implemente a solução utilizando 2 buscas e o cálculo de profundidade.

Accepted

Tempo: 0.00s — Memória: 0 KB

Nos vemos na próxima aula.